

Instinto

Su misión, si deciden aceptarla, es construir una aplicación en Python capaz de leer criaturas programadas en un lenguaje que denominaremos: *Instinct*. Las criaturas se colocan sobre un terreno y se dejan a su suerte: pueden pelear, alimentarse, reproducirse, morir. El usuario de la aplicación jugará a ser Dios y observará desde arriba cómo se comportan.

1. La idea general

El proyecto va a tener tres partes:

1. *Un compilador e intérprete* del lenguaje *Instinct*. Deben ser capaces de leer un archivo de texto (con extensión .ins), comprobar que esté léxica, sintáctica y semánticamente bien escrito y pasarle la interpretación (de alguna manera) al simulador. Los errores deben reportarse con número de línea.
2. *Un mundo simulado*: una grilla de dos dimensiones que representa el terreno. Sobre él, criaturas y objetos. El mundo, una vez iniciada la simulación, avanza por turnos discretos llamados *ticks*. Hay leyes naturales que se deben respetar: la vida baja un punto cada tick, quien llega a 0 muere, quien es atacado pierde vida, y aquel cuya edad llegue a su esperanza de vida, muere.
3. *Una aplicación con interfaz visual*: el usuario carga las criaturas, terrenos, objetos y mapas que existan en el repositorio, escoge un mapa disponible, coloca criaturas donde desee, da play y observa la simulación mientras acelera, ralentiza o pausa el tiempo.

La base que se describe en las secciones 2 a 4 es común para todos: el lenguaje *Instinct* sin extender, las reglas mínimas del mundo y las funciones mínimas de la aplicación. Cualquier criatura, terreno, objeto o mapa escrito según esta base debe funcionar en el proyecto de cualquier estudiante. Presten atención a esto, pues se comprobará durante la defensa. Los profesores tendrán ejemplos que deben correr en sus aplicaciones. Si la defensa empieza con una explosión catastrófica... uff, vamos mal.

Sobre la base, ustedes como estudiantes tienen varias libertades: la temática (animales, robots, bacterias, zombies, marmotas asesinas, extraterrestres de Star Wars, Titanes caníbales, en fin, lo que les guste), la biblioteca gráfica, el diseño de la interfaz, y sobre todo (redoble de tambores) las *extensiones*: nuevas acciones, funciones y percepciones del lenguaje, nuevos atributos y mecánicas de las criaturas, nuevos terrenos y objetos con comportamiento propio, herramientas dentro de la aplicación.

Instinct ofrece varias acciones con la forma de una llamada a función. Estas son: `move(...)`, `attack(...)`, `consume(...)`, `reproduce(...)`, `roar(...)`, `say(...)`, `wait(...)`. Cada una tiene un efecto mecánico definido. La criatura (el código que representa su instinto) decide *cuándo* la ejecuta, *sobre qué* y *con qué cantidades*, porque esas cantidades son argumentos. Lo que en la base se entiende como *consumir* puede ser comer, hacer fotosíntesis, masticar piedras o bailar para alimentarse de la felicidad del objetivo (xD). "Reproducirse" puede ser parir, dividirse en dos o fabricar una copia a un paso de distancia mientras el original muere (Esto sería una criatura que camina dejando un rastro de cadáveres jajajaj). La mecánica es la misma, la interpretación la pone cada estudiante con su temática. Y como las siete acciones comparten la misma forma, añadir una acción nueva al lenguaje debería ser escribir una clase más, sin tocar ninguna de las que ya existen.

Una criatura es un ciclo lleno de condicionales anidadas que, en cada situación, decide qué hacer a partir de lo que percibe: sobre sí misma, sobre el mapa, sobre lo que tiene a su alrededor. Un

ejemplo tonto: "si la cantidad de criaturas de mi facción en el mapa supera N, rujo". En este caso la raza se condena porque nuestras brillantes criaturas hipotéticas no van a parar de gritar hasta que alguien de su especie muera. Cuando varias especies así conviven en un mismo mapa aparecen comportamientos que nadie programó explícitamente. Ese va a ser el efecto WAO.

2. El lenguaje Instinct

Cada especie de criatura se define en un archivo de texto con extensión `.ins`. Un archivo tiene dos partes: una *cabecera* con los atributos de la especie y un *cuerpo* con su comportamiento.

2.1 Un ejemplo completo

```
# uruk.ins
creature Uruk
faction isengard
health 80
vision 6
lifespan 400

start:
    if health < 20 goto flee
    if enemy_dist == 1 goto bite
    if enemy_dist > 0 goto hunt
    if health > 70 and allies_near < 2 goto breed
    if allies_near > 3 goto celebrate
wander:
    move(random % 3 - 1, random % 3 - 1, 1)
    goto start

bite:
    consume(enemy_dx, enemy_dy, 15, 15)
    goto start

hunt:
    move(enemy_dx, enemy_dy, 2)
    goto start

flee:
    move(-enemy_dx, -enemy_dy, 3)
    goto start

breed:
    # bred in the pits of Isengard
    if see(x, y + 1) != GROUND goto wander
    reproduce(0, 1, 30)
    goto start

celebrate:
    say("meat is back on the menu")
    roar("we are the fighting Uruk-hai")
    goto start
```

2.2 La cabecera

Todo lo que aparece antes de la etiqueta `start:` es la cabecera. Cada línea tiene la forma `id valor`. La primera línea no vacía debe ser `creature Name`, y los otros cuatro ids pueden ir en cualquier orden. Las cinco son obligatorias: si falta una, el archivo no compila.

id	Significado
creature Name	Nombre de la especie. Identifica el archivo en la aplicación.
faction Name	Facción. Dos criaturas son <i>aliadas</i> si sus facciones son la misma cadena de texto y <i>enemigas</i> en otro caso. Varias especies pueden compartir facción.
health N	Vida inicial. No hay vida máxima: se puede acumular tanta como se consiga. Baja 1 en cada tick y al llegar a 0 la criatura muere. Debe ser mayor que 0.
vision N	Radio (en casillas) dentro del cual la criatura percibe lo que hay a su alrededor. Limita también hasta dónde puede mirar con see y name . Debe ser mayor o igual que 1.
lifespan N	Esperanza de vida en ticks. Cuando la edad alcanza este valor, la criatura muere de vieja. Debe ser mayor que 0.

Todas las criaturas ocupan exactamente una casilla.

2.3 El cuerpo

El cuerpo empieza en la etiqueta obligatoria **start:** y es una secuencia de líneas. Cada línea es *una* de estas cosas:

Línea	Qué hace
# text	Comentario. Se ignora. También al final de cualquier línea.
name:	Etiqueta. Marca un punto al que se puede saltar. No consume turno.
goto name	Salto incondicional a la etiqueta.
if expr goto name	Salto condicional: salta si la expresión es verdadera.
name = expr	Asignación. Crea la variable si no existía. No se puede asignar a una percepción ni a una constante.
action(arg, arg, ...)	<i>Acción</i> . Una llamada sola en su línea, con o sin argumentos. Las acciones básicas están en la tabla siguiente.

Las únicas palabras reservadas del lenguaje son **if**, **goto**, **and**, **or** y **not**. Todo lo demás (acciones, funciones, percepciones, constantes) son nombres que el intérprete resuelve contra su catálogo de acciones, funciones y percepciones, y ese catálogo es lo que ustedes, en casode atreverse, van a extender. Los nombres empiezan por letra o guion bajo y siguen con letras, dígitos o guiones bajos. Se distingue entre mayúsculas y minúsculas. La indentación es libre y no significa nada.

2.4 Las acciones básicas

Una acción se escribe sola en su línea, como una llamada a función. Todos los proyectos deben tener estas siete implementadas, con exactamente estos nombres, argumentos y efectos. Las direcciones se dan con dos números **dx** y **dy**, cada uno en $\{-1, 0, 1\}$: **dx** = 1 es hacia la derecha, **dy** = 1 es hacia abajo, y ambos distintos de 0 es una diagonal.

Acción	Efecto
wait(n)	No hace nada durante n turnos seguidos. Si n es menor que 1 cuenta como 1.
move(dx, dy, speed)	Avanza hasta speed casillas en la dirección (dx , dy), una a una. Se detiene antes de la primera casilla que no pueda pisar (fuera del mapa, con objeto o con criatura). Consume un solo turno aunque avance varias casillas.

Acción	Efecto
<code>attack(dx, dy, damage)</code>	Golpea lo que haya en la casilla adyacente en dirección <code>(dx, dy)</code> . Si hay una criatura, pierde damage de vida. Si hay un objeto, pierde damage de reserva. Si solo hay terreno, el terreno pierde damage de reserva. Con <code>(0, 0)</code> el objetivo es el terreno bajo la propia criatura. Siempre cuerpo a cuerpo, distancia 1.
<code>consume(dx, dy, damage, heal)</code>	Igual que <code>attack</code> , y además la criatura gana heal puntos de vida. Sin tope: el lenguaje no relaciona heal con damage ni con lo que el objetivo tenía. Poner límites sensatos es responsabilidad de quien programa la criatura.
<code>reproduce(dx, dy, health)</code>	Crea una criatura de la misma especie (mismo archivo, misma facción) en la casilla adyacente en dirección <code>(dx, dy)</code> , siempre a distancia 1. Requiere que esa casilla tenga solo terreno y que la madre tenga más de health puntos de vida. La madre pierde health de vida y la cría nace con esa vida, edad 0, sin ninguna variable (no hereda nada de lo que la madre tuviera guardado: si la madre recordaba su casa en <code>home_x</code> , la hija ni sabe que existe) y con su programa en <code>start:.</code> Si algo falla, considérenlo un aborto natural.
<code>roar(value)</code>	Emite un rugido. Las criaturas que lo oigan pueden leer su texto en el tick siguiente, y la interfaz visual debe mostrarlo de alguna manera.
<code>say(value)</code>	Muestra un texto sobre la criatura o en el registro de la aplicación. Es la única acción que <i>no</i> consume turno. Si sincronizamos bien, pudiéramos lograr que un grupo de criaturas canten We Are The World.

Toda acción salvo `say` consume el turno, tenga éxito o no. Las cantidades (`damage`, `heal`, `speed`, `health`) son decisión de quien programa la criatura: el lenguaje no impone límites, la simulación se encarga de premiar o castigar las decisiones instintivas.

2.5 Expresiones

En el lenguaje base hay dos tipos: *números* (enteros) y *texto*. Una expresión se forma con:

- Literales enteros: 0, 42, -7.
- Literales de texto entre comillas dobles: "grass", "AW000", "".
- Variables propias, percepciones y constantes (vean la sección 2.6).
- Llamadas a funciones reservadas, con argumentos entre paréntesis separados por comas. En la base hay dos: `see(x, y)` y `name(x, y)`.
- Aritmética: +, -, *, / (división entera, como // en Python), % (resto), menos unario, paréntesis.
- Comparaciones: <, <=, >, >=, ==, !=. Valen 1 si son ciertas y 0 si no.
- Lógica: and, or, not. Devuelven 0 o 1.

Aquí les va la precedencia (de mayor a menor) que siempre causa discordia: paréntesis y llamadas, menos unario y not, luego * / %, luego + -, luego comparaciones, luego and, luego or. Está diseñada como la de Python, así que `if health < 20 and enemy_dist < 3 goto x` significa lo que uno espera de forma intuitiva.

En una variable se puede guardar un entero o un texto. Las variables pueden, igual que en Python, cambiar de tipo cuando se les vuelve a asignar un valor. Ahora, todos los operadores, salvo los lógicos, deben cumplir que sus dos lados tengan el mismo tipo o de lo contrario lanzan error en tiempo de ejecución. El texto solo admite == y != como operaciones. La aritmética y las comparaciones de orden (<, <=, >, >=) solo aceptan números. Los lógicos son la excepción, porque no comparan nada: son *falsos* el entero 0 y el texto vacío "", y verdadero todo lo demás, así que and, or y not

se tragan cualquier combinación de tipos sin protestar. Las acciones que esperan números (*move*, *attack*, *consume*, *reproduce*, *wait*) dan también error de ejecución si reciben un texto, mientras que *roar* y *say* aceptan cualquier valor (si reciben un número, pues muestran esa cifra).

2.6 Percepciones, constantes y las funciones *see* y *name*

Las criaturas son capaces de ver el mundo a su alrededor de dos maneras. Con percepciones y con funciones (inicialmente sólo *see* y *name*). Las *percepciones* son variables de solo lectura que se deben rellenar al comienzo de cada turno. Ojo: el simulador debe encargarse de eso. Así en cada momento, la criatura sabe información de sí misma y de lo que tiene cerca. Las funciones *see(x, y)* y *name(x, y)* le permiten además preguntar por cualquier casilla dentro de su radio de visión, con coordenadas del canvas.

see(x, y) devuelve una de estas cinco constantes:

Constante	Valor	Significado
NONE	-1	La casilla está fuera del radio de visión de quien pregunta (a distancia mayor que <i>vision</i>). No se sabe qué hay. Se comprueba antes que todo lo demás.
GROUND	0	En la casilla solo hay terreno. Se puede pisar.
OBJECT	1	Hay un objeto (una roca, por ejemplo). No se puede pisar. Las coordenadas fuera del mapa pero dentro del radio de visión también devuelven OBJECT : el borde se comporta como un muro indestructible.
ALLY	2	Hay una criatura de la misma facción que quien pregunta (la propia casilla devuelve ALLY).
ENEMY	3	Hay una criatura de otra facción.

Por otro lado *name(x, y)* devuelve el nombre, como un objeto de tipo texto, de lo que actualmente está en esa casilla: el de la especie si hay una criatura, el del objeto si hay un objeto, y el del terreno si no hay nada. Devuelve el texto vacío "" si la casilla está fuera del radio de visión o fuera del mapa. Así una criatura puede distinguir la hierba del agua, un cambolo de un árbol, sin que el lenguaje deba conocer a priori los nombres que cada estudiante se invente.

Percepciones sobre sí misma

Percepción	Significado
<i>health</i>	Vida actual.
<i>age, lifespan</i>	Edad en ticks y esperanza de vida. La edad nunca es menor que 0.
<i>vision</i>	El atributo de la cabecera.
<i>species, faction</i>	Texto: el nombre de su especie y el de su facción.
<i>x, y</i>	Posición en el mapa. La casilla superior izquierda es (0, 0), <i>x</i> crece hacia la derecha e <i>y</i> hacia abajo.
<i>terrain_here</i>	Texto: nombre del terreno bajo la criatura.
<i>reserve_here</i>	Reserva del terreno bajo la criatura.

Percepciones sobre el mundo

Percepción	Significado
<i>width, height</i>	Tamaño del mapa en casillas.
<i>tick</i>	Número del tick actual.
<i>total_creatures, total_allies, total_enemies</i>	Cuántas criaturas hay vivas en todo el mapa (sin contarse a sí misma).

Percepción	Significado
<code>random</code>	Entero aleatorio entre 0 y 99, distinto cada vez que se lee. Es la única fuente de aleatoriedad del lenguaje.

Ojo con `random`: se lee de verdad cada vez que aparece, así que `random - random` casi nunca da 0 (las probabilidades de esto son bien pero bien bajitas). Y toda la aleatoriedad del simulador (que es esta percepción y el sorteo del orden de los turnos) debería salir de un mismo generador con una semilla, para que una simulación se pueda repetir idéntica si se guarda esa semilla.

Percepciones sobre lo cercano (dentro del radio `vision`. La distancia entre dos casillas es el mayor de los dos desplazamientos en `x` y en `y`, de modo que las ocho casillas vecinas están a distancia 1)

Percepción	Significado
<code>allies_near</code> , <code>enemies_near</code> , <code>objects_near</code>	Cuántos hay dentro del radio de visión.
<code>enemy_dist</code> , <code>enemy_dx</code> , <code>enemy_dy</code>	Distancia al enemigo más cercano y dirección hacia él, con <code>dx</code> y <code>dy</code> en <code>{-1, 0, 1}</code> , listos para pasarlos a <code>move</code> , <code>attack</code> o <code>consume</code> . Si no hay ninguno visible, <code>enemy_dist</code> vale -1 y las direcciones 0.
<code>ally_dist</code> , <code>ally_dx</code> , <code>ally_dy</code>	Lo mismo para el aliado más cercano.
<code>object_dist</code> , <code>object_dx</code> , <code>object_dy</code>	Lo mismo para el objeto más cercano.
<code>roars_near</code>	Cuántas criaturas rugieron en el tick anterior dentro del radio de visión.
<code>last_roar</code>	Texto: lo que decía el último de esos rugidos, o "" si no oyó ninguno.

Cuando hay varios candidatos a la misma distancia se elige uno cualquiera, pero hasta que la situación no cambie (una se acerque más, el objetivo salga del rango...) se escogerá siempre la misma.

Noten que con `see` y `name` las criaturas pueden ver lo que las percepciones no les cuentan si recorren todas las casillas dentro del radio de visión.

Este conjunto de percepciones de arriba es el mínimo obligatorio del lenguaje, siéntanse libres de añadir todas las que quieran.

2.7 Cómo se ejecuta una criatura

Léanse esto 3 o 4 veces, pues es importante conocer las leyes naturales que nos rodean (en este caso que rodean a sus bichos):

1. El mundo avanza en *ticks*. En cada tick, cada criatura viva recibe *un turno*. El orden de los turnos se sortea al azar en cada tick.
2. Al comenzar su turno, el simulador actualiza todas las percepciones de la criatura.
3. La criatura ejecuta instrucciones a partir de donde se quedó en el turno anterior (la primera vez, desde `start`:). Asignaciones, saltos, etiquetas y `say` son *gratis*: no consumen turno.
4. El turno termina en cuanto ejecuta *una acción* (cualquiera salvo `say`). La acción se aplica al mundo y la criatura recuerda en qué instrucción se quedó. Si la acción fue `wait(n)`, la criatura además deja pasar los siguientes `n - 1` turnos sin ejecutar nada. Dormir no la protege de nada: el mundo le sigue quitando vida y sumándole edad en cada tick, la pueden atacar y se puede morir de vieja. Cuando despierta, sus percepciones se rellenan como en cualquier otro turno.
5. Una criatura ejecuta como mucho **100 líneas por turno**. Si llega a esa cuenta sin haber hecho ninguna acción, el turno termina igual (como si hubiera hecho `wait(1)`). Así un ciclo mal escrito no congela la aplicación.

6. Si la ejecución llega al final del archivo, continúa desde `start:` en el próximo turno.
7. Las variables propias conservan su valor entre turnos. Eso permite tener memoria: contar cuántas veces se ha peleado, recordar hacia dónde iba, guardar la posición de su casa.
8. Una criatura recién nacida recibe su primer turno en el tick siguiente al de su nacimiento.

2.8 Errores

Instinct, como suele pasar con los lenguajes de programación, tiene dos tipos de errores:

- *Errores en tiempo de compilación:* línea que no encaja en ninguna de las formas válidas, etiqueta usada que no existe, etiqueta duplicada, falta alguna de las cinco claves de la cabecera o falta `start:`, valor inválido en la cabecera, expresión mal formada, asignación a una percepción o constante, acción o función con nombre desconocido, acción o función con un número de argumentos distinto del que espera. Al tratar de cargar el archivo en la aplicación, esta lo rechaza, informa *qué* pasó y *en qué línea*, y sigue funcionando con las demás criaturas. Este error se calcula sin necesidad de simular nada.
- *Errores en tiempo de ejecución:* división por cero, variable que se lee antes de asignarse, `dx` o `dy` fuera de `{-1, 0, 1}`, mezclar un entero y un texto en un mismo operador, y usar texto donde hace falta un número. Cuando esto pasa, a la criatura le da un ictus y se muere ahí mismo. El mensaje de error, con su línea, aparece en los logs de la aplicación.

Base obligatoria del lenguaje -> Todo lo descrito en esta sección: cabecera con sus cinco claves, las formas de línea, las siete acciones básicas con sus argumentos exactos, expresiones con enteros y texto, las funciones `see` y `name`, las cinco constantes, todas las percepciones listadas, el modelo de turnos y el tratamiento de errores.

Espacio para extender -> Acciones nuevas (`shoot(dx, dy, range, damage)`, `heal(dx, dy, amount)`, `rejuvenate(amount)`, `build(dx, dy, "Wall")`), funciones reservadas nuevas (`reserve(x, y)`, `health_at(x, y)`, `distance(x1, y1, x2, y2)`), percepciones nuevas, estructuras de control de alto nivel (`while`, `if / else / end`) que su compilador traduzca a saltos, subrutinas con `call` y `return`, concatenación de texto, números reales, listas.

3. El mundo

3.1 Casillas, terreno, objetos y criaturas

El mundo es plano, a pesar de lo que dicen los terrarredondistas. Es una rejilla rectangular de casillas. En cada casilla puede haber hasta dos elementos: siempre hay *terreno*, y sobre el terreno puede haber *un objeto* o *una criatura*, nunca los dos a la vez.

Elemento	Se pisa	Reserva	Qué pasa al llegar a 0
Terreno	Sí	Empieza llena y se regenera regen por tick hasta resource_max	Nada, sigue ahí con reserva 0 hasta que se regenera
Objeto	No	Empieza en resource_max y <i>no</i> se regenera	Desaparece y la casilla queda con solo terreno
Criatura	No	Su vida	Muere y desaparece

La *reserva* es lo que `attack` y `consume` le quitan al terreno y a los objetos. Consumir de la hierba es pastar o hacer fotosíntesis. Consumir de una roca es minarla. Lo que signifique cada cosa depende de la temática de cada proyecto.

3.2 Terrenos y objetos

Los terrenos y los objetos no están fijados en el código: se definen en archivos que la aplicación carga al arrancar, igualito a las criaturas, lo que los hace muchísimo más sencillos (no se preocupen, no va a aparecer de pronto un lenguaje salvaje para definir una piedra). Un terreno es cualquier cosa que declare `resource_max` y `regen`. Un objeto es cualquier cosa que declare solo `resource_max`. Esa es toda la diferencia entre los dos.

Un terreno vive en un archivo `.te` dentro del directorio `terrains/`:

```
# grass.te
terrain Grass
char .
resource_max 100
regen 2
```

Un objeto vive en un archivo `.ob` dentro del directorio `objects/`:

```
# rock.ob
object Rock
char #
resource_max 100
```

La clave `char` es el carácter con el que ese elemento aparece en los mapas en caso de que la aplicación sea de consola. Distintos visuales pueden darle una vuelta a esto. Debe ser distinto para cada terreno y cada objeto cargado. El nombre (`Grass`, `Rock`) es lo que devuelve la función `name(x, y)` y es también lo que la interfaz mágica hipotética puede usar para decidir cómo dibujarlo (esto quedaría volao, sólo digo).

La base son la hierba y la roca, exactamente con los valores de arriba. Los estudiantes pueden añadir todos los terrenos y objetos que quieran (`water.te`, `lava.te`, `tree.ob`, `wall.ob`), y sus extensiones pueden darles comportamientos que la base no tiene. Lo que no pueden es romper los dos básicos ni dejar de cargar archivos que cumplan con esto cuando venga la defensa.

3.3 Los mapas

Un mapa vive en un archivo `.map` dentro del directorio `maps/`. La primera línea es su nombre, y cada línea siguiente es una fila de casillas, todas del mismo largo. Cada carácter es el `char` de un terreno o de un objeto: si es el de un objeto, la casilla lleva ese objeto sobre el primer terreno declarado.

```
Plain with rocks
.....
...##.....
...##....#.....
.....#....###..
.....#.....
.#.....
.#.....##.....
```

Si un mapa usa un `char` que ningún terreno ni objeto cargado declara, el mapa se rechaza con un mensaje que dice cuál es el `char` y en qué línea aparece, y la aplicación sigue funcionando con los demás. Aquí evitamos los errores catastróficos: vivimos en Cuba, así que tenemos resiliencia.

Se entregarán varias criaturas, terrenos, objetos y mapas hechos por los profesores. Sus aplicaciones deben poder cargarlos todos, y ustedes pueden crear más de lo que quieran, siempre que esté dentro de los límites morales (no quiero genocidios).

3.4 Reglas de la simulación

Durante el turno de una criatura se aplican los efectos de la acción ejecutada, tal como se describen en la sección 2.4. Dos precisiones:

- El objetivo de `attack`, `consume` y `reproduce` es siempre una casilla adyacente (con diagonales incluidas). Fuera del mapa no hay a qué darle un golpe: la acción no hace nada.
- Si un `attack` o un `consume` deja a una criatura con vida 0 o menos, se queda tesa ahí mismo (muere) y su casilla pasa a ser sólo el terreno que tiene debajo.

Al final de cada tick, después de todos los turnos:

1. Toda criatura pierde 1 punto de vida y suma 1 a su edad. La edad nunca baja de 0, ni siquiera si una extensión permite rejuvenecer.
2. Toda criatura con vida menor o igual a 0, o con edad mayor o igual que `lifespan`, muere y desaparece del mapa.
3. Todo terreno recupera su `regen` de reserva, sin pasar de su `resource_max`.
4. Todo objeto con reserva 0 o menos desaparece.

Les arrojo algunas ideas para extender: terrenos con efecto propio (agua que no se pisa, lava que quita vida, arena que reduce la velocidad, hielo que hace resbalar), objetos con comportamiento (árboles que se regeneran, comida que aparece al azar, muros indestructibles, cadáveres que deja una criatura al morir), atributos nuevos en los archivos `.te` y `.ob`, energía además de la vida, envejecimiento que degrada atributos, herencia y mutación en la reproducción, ciclo de día y noche, generación aleatoria de mapas.

Pero como dijo el Tío Ben, un gran poder conlleva una gran responsabilidad. Estas extensiones no pueden romper el comportamiento base.

4. La aplicación

La aplicación gráfica es libre, no los vamos a coaccionar. Investiguen.

4.1 Flujo de uso

1. Primero se arranca: la aplicación lee los archivos `.te` de `terrains/`, los `.ob` de `objects/`, los `.map` de `maps/` y los `.ins` de `creatures/`, en ese orden. Lo que carga bien queda disponible. Lo que no, se reporta con su error y su línea, y la aplicación sigue funcionando.
2. Luego se prepara la simulación. El usuario escoge un mapa, y luego coloca criaturas sobre él: elige una especie y señala una casilla. Debe poder colocar varias criaturas de la misma especie y quitar las que coloque por error. No se puede colocar sobre un objeto ni sobre otra criatura.
3. Empezamos la simulación. El usuario pulsa play. A partir de ahí la única influencia posible sobre el mundo es la velocidad del tiempo: pausa, avanzar un solo tick, y al menos tres velocidades distintas (por ejemplo 2, 10 y 30 ticks por segundo). No se pueden añadir, quitar ni tocar criaturas.
4. También está la opción de reiniciar. Un botón devuelve a la fase de preparación.

Debe haber un registro (logs) con los errores de carga al arrancar, los errores de ejecución, y lo que sea que quieran mostrar.

Algunas ideas que les arrojo para extender: inspector de criatura al hacer clic (sus variables, sus percepciones, la línea que está ejecutando), gráficas de población en tiempo real, guardar y cargar el estado de una simulación, repetición de una simulación con la misma semilla, editor de código dentro de la aplicación con recarga sin reiniciar, editor y generador de mapas, sprites y animaciones.

5. El diseño que esperamos

Esto es un curso de Programación Orientada a Objetos, así que el proyecto no se evalúa solo por si corre. Se evalúa por cómo está armado por dentro.

Un intérprete es probablemente el mejor ejemplo que existe de polimorfismo útil, así que aprovéchelo. Lo que no queremos ver es una cadena de `if tipo == "move": ... elif tipo == "attack": ...` de doscientas líneas. Eso funciona, sí, y es exactamente lo primero que les va a proponer cualquier IA si le piden "un intérprete en Python". Pero es lo contrario de lo que estamos enseñando.

Las jerarquías que esperamos (sugerencias de nuestra parte):

- *Las acciones.* Una clase abstracta y una subclase por cada una de las siete. Cada subclase sabe validar sus argumentos y aplicar su efecto sobre el mundo.
- *Las expresiones.* El AST es una jerarquía: literales, variables, operaciones, llamadas. Cada nodo sabe evaluarse solo.
- *Lo que vive en el mundo.* Terreno, objetos y criaturas comparten cosas (ocupan una casilla, tienen una reserva que se les puede quitar a golpes) y se diferencian en otras. Eso es (redoble de tambores)... una jerarquía.
- *Las percepciones.* Son muchas y todas se ven igual desde el intérprete. Resuélvanlas de forma inteligente, no con un `if nombre == ...` por cada una de las treinta.
- *El mundo y la interfaz.* Al simulador le da igual si hay o no una interfaz. Si no pueden correr una simulación completa desde la terminal, sin abrir visual, está mal separado.

Sobre el alcance: queremos un intérprete que recorra el AST y ya, no una máquina virtual con bytecode. Si quieren entrar en ese tipo de cosas, adelante (de hecho como profesores siempre nos va a alegrar que ustedes vayan un poco más allá), pero está fuera del requisito base. Con la aplicación pasa igual: una ventana con la rejilla, las criaturas, el registro y los controles de tiempo cumple con la base. Los sprites, las animaciones y el inspector son extensión (espectacular), no requisito.

6. Evaluación

Implementar el proyecto base descrito en las secciones 2 a 4 equivale a 3 puntos, la nota mínima de aprobado, siempre que el proyecto no explote cuando el profe ponga sus ejemplos y no se haga un desastre durante la defensa.

De 3 para arriba, lo que decide la nota es sobre todo la sección 5. Un proyecto que cumple la base con el código hecho un desastre se queda en 3. Para 4 o 5 hay que hacer cosas extra, entre las que están, pero sin limitarse a: tener una arquitectura sólida, extender el lenguaje, hacer una buena aplicación visual y, por supuesto, hacer una buena defensa del proyecto.

Que sus aplicaciones puedan cargar los ejemplos de los profesores es la razón por la que la base es igual para todos. Sus extensiones se añaden a la base, nunca la sustituyen: si `if` pasa a exigir un bloque y deja de aceptar `if expr goto label`, si `move` pasa a recibir dos argumentos en vez de tres,

o si la aplicación no sabe dibujar un terreno cuyo nombre no conocía... estamos mal.

La defensa se hace presencialmente, de forma *individual*, con uno o más profesores. Hay que conocer bien el propio código, porque se harán preguntas al estilo de (pero no limitadas a):

- *Qué hace esta función.*
- *Por qué lo implementaste así y no de esta otra forma.*
- *Te elimino un fragmento de código y lo vuelves a implementar delante de mí.*

Sobre la IA: úsenla, no vamos a hacer de cuenta que eso no existe. Lo que evaluamos no es quién escribió el código, es quién lo entiende, y eso se mide de una sola forma: lo que no puedan explicar y rehacer delante de nosotros no cuenta como suyo, lo haya escrito quien lo haya escrito. Una IA les arma un intérprete en diez minutos. Lo que no les puede dar es saber por qué está armado así, que es exactamente lo que se pregunta en la defensa.

Una buena defensa puede subir la nota de un proyecto aprobado. Una mala defensa también puede bajarla hasta 2 puntos, aunque el código entregado funcione.

Anexo: Terrenos, objetos y criaturas de ejemplo

```
# grass.te
terrain Grass
char .
resource_max 100
regen 2
```

```
# rock.ob
object Rock
char #
resource_max 100
```

Un hobbit: pacífico, come en cuanto puede, huye de todo, busca compañía y llena la Comarca de hijos.

```
# hobbit.ins
creature Hobbit
faction shire
health 60
vision 5
lifespan 300

start:
    if enemy_dist >= 0 goto flee
    if health > 45 and total_allies < 20 goto breed
    if reserve_here > 0 and health < 80 goto second_breakfast
    if ally_dist > 1 goto go_ally
    wait(1)
    goto start
flee:
    say("they are taking us to Isengard")
    move(-enemy_dx, -enemy_dy, 2)
    goto start
second_breakfast:
    consume(0, 0, 10, 10)
    goto start
```

```

go_ally:
    move(ally_dx, ally_dy, 1)
    goto start
breed:
    if see(x + 1, y) == GROUND goto breed_right
    if see(x - 1, y) == GROUND goto breed_left
    wait(1)
    goto start
breed_right:
    reproduce(1, 0, 20)
    goto start
breed_left:
    reproduce(-1, 0, 20)
    goto start

```

Un ent: no se mueve de su bosque, se alimenta por las raíces, avisa antes de pelear y vuelve a su sitio si algo lo aparta de él.

```

# ent.ins
creature Ent
faction fangorn
health 150
vision 4
lifespan 1000

start:
    home_x = x
    home_y = y
watch:
    if enemy_dist == 1 goto hit
    if enemies_near > 0 and roars_near == 0 goto warn
    if x != home_x or y != home_y goto go_home
    if health < 150 goto roots
    wait(1)
    goto watch
hit:
    attack(enemy_dx, enemy_dy, 12)
    goto watch
warn:
    roar("hoom, hom, do not be hasty")
    goto watch
roots:
    consume(0, 0, 3, 3)
    goto watch
go_home:
    dx = (home_x > x) - (home_x < x)
    dy = (home_y > y) - (home_y < y)
    move(dx, dy, 1)
    goto watch

```

Un enano: camina sin rumbo, mina toda roca que encuentra, pide auxilio cuando lo acorralan y acude al grito de los suyos.

```

# dwarf.ins
creature Dwarf
faction erebor
health 100
vision 3
lifespan 600

start:
    if last_roar == "khazad ai-menu" goto rescue
    if object_dist == 1 and health < 100 goto mine

```

```
dx = random % 3 - 1
dy = random % 3 - 1
steps = 0
walk:
    if steps >= 5 goto start
    if enemy_dist == 1 goto shout
    move(dx, dy, 1)
    steps = steps + 1
    goto walk
mine:
    # never too greedily and too deep
    if name(x + object_dx, y + object_dy) != "Rock" goto start
    consume(object_dx, object_dy, 20, 20)
    goto start
shout:
    roar("khazad ai-menu")
    goto start
rescue:
    say("and my axe")
    move(ally_dx, ally_dy, 2)
    goto start
```

Dos detalles de los ejemplos. En el ent, la expresión $(\text{home_x} > x) - (\text{home_x} < x)$ vale 1, 0 o -1 según haya que ir a la derecha, quedarse o ir a la izquierda: es la forma de calcular el signo de una diferencia con las herramientas de la base. En el hobbit, el `say` antes de huir no cuesta nada, porque es la única acción que no consume el turno.